SCHOOLOS • ENGINEERING GUIDE

Complete Postman API Testing & Verification Guide

Step-by-step verification manual for Authentication, User Administration, and Assignments Modules

BACKEND ENGINE
Node.js, Express.js, TypeScript

DATABASE & ORM
PostgreSQL (Supabase) + Prisma ORM

DEFAULT SERVER PORT
`http://localhost:4000`

API BASE ROUTE
`/api/v1`

ACTIVE MODULES COVERED
Auth (Dev 1), Users/Admin (Dev 1), Assignments (Dev 3)

DATE GENERATED
October 2026

Pre-Seeded Credentials for Immediate Testing:

- **Teacher:** `teacher@schoolos.com` | Password: `Teacher123!`
- **Student:** `student@schoolos.com` | Password: `Student123!`
- **Admin:** `admin@schoolos.com` | Password: `devinton@2026`
- **Demo Course ID:** `00000000-0000-0000-0000-000000000001` (Algorithms & Data Structures)

1. Server Startup & Postman Setup

Before executing API calls, confirm the backend server is running cleanly:

```
cd D:\Projects\SchoolOS_Project\school-os\backend
npm run dev
```

The console must output: `Server running on http://localhost:4000`.

Recommended Postman Environment Variables

In Postman, create an Environment (e.g. "*SchoolOS Local*") with the following key-value pairs:

Variable	Initial Value	Description
`baseUrl`	`http://localhost:4000`	Base server URL
`accessToken`	*(Leave blank initially)*	JWT access token copied after login
`refreshToken`	*(Leave blank initially)*	Refresh token for rotating sessions
`courseId`	`00000000-0000-0000-0000-000000000001`	Pre-seeded demo course ID
`assignmentId`	*(Filled after creating assignment)*	ID returned from POST /assignments
`submissionId`	*(Filled after submitting work)*	ID returned from POST /submissions

> **Header Configuration for Protected Routes:**
> In Postman, under the **Authorization** tab, select type **Bearer Token** and enter `{{accessToken}}` (or manually add the header `Authorization: Bearer <token>`).

2. System Health & Authentication APIs (Developer 1)

GET `/health` — Public

Verifies server availability and routing health.

EXPECTED RESPONSE (200 OK)

```
{
  "status": "ok"
}
```

POST `/api/v1/auth/signup` — Public (Creates Student)

Registers a new user account. Defaults strictly to STUDENT role.

REQUEST BODY (APPLICATION/JSON)

```
{
  "firstName": "Alex",
  "lastName": "Rivera",
  "email": "alex.rivera@example.com",
  "password": "Password123!"
}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Account created successfully",
  "data": {
    "id": "c1f7a4e2-8821-4d92-9118-971c26fca041",
    "firstName": "Alex",
    "lastName": "Rivera",
    "email": "alex.rivera@example.com",
    "role": "STUDENT"
  }
}
```

POST /api/v1/auth/login

Public

Authenticates credentials and returns a short-lived JWT access token and a refresh token.

REQUEST BODY (APPLICATION/JSON)

```
{
  "email": "teacher@schoolos.com",
  "password": "Teacher123!"
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
      "id": "0c621547-8988-4acc-b15e-939a2a4174e2",
      "firstName": "Professor",
      "lastName": "Smith",
      "email": "teacher@schoolos.com",
      "role": "TEACHER"
    },
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
    "refreshToken": "935a32957bb4482536e1dbd3b81..."
  }
}
```

GET /api/v1/auth/me

Protected (Bearer Token)

Fetches the authenticated user's profile from the JWT payload.

HEADERS

```
Authorization: Bearer {{accessToken}}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "data": {
    "id": "0c621547-8988-4acc-b15e-939a2a4174e2",
    "firstName": "Professor",
    "lastName": "Smith",
    "email": "teacher@schoolos.com",
    "role": "TEACHER",
    "bio": "Computer Science Faculty Lead"
  }
}
```

PATCH `/api/v1/auth/me` — Protected (Bearer Token)

Updates self-profile fields: `firstName`, `lastName`, `bio`, `avatarUrl`.

REQUEST BODY (APPLICATION/JSON)

```
{
  "firstName": "Alexander",
  "bio": "Software Systems & Security Researcher"
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": "c1f7a4e2-...",
    "firstName": "Alexander",
    "lastName": "Rivera",
    "bio": "Software Systems & Security Researcher"
  }
}
```

POST `/api/v1/auth/refresh` — Public

Exchanges an active refresh token for a brand new access token.

REQUEST BODY

```
{
  "refreshToken": "{{refreshToken}}"
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Tokens refreshed",
  "data": {
    "accessToken": "eyJhbGciOi..."
  }
}
```

POST `/api/v1/auth/logout` Public

Revokes the refresh token in PostgreSQL, terminating the device session.

REQUEST BODY

```
{
  "refreshToken": "{{refreshToken}}"
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

3. User Administration APIs (Developer 1)

> **Admin Authorization Required:** All endpoints in this section require an access token belonging to an account with `role: "ADMIN"` (such as `admin@schoolos.com`).

GET `/api/v1/users` — ADMIN Only

Returns a list of all registered users. Optional filter by query parameter: `?role=TEACHER` or `?role=STUDENT`.

HEADERS

```
Authorization: Bearer {{adminToken}}
```

EXPECTED RESPONSE (200 OK)

```
[
  {
    "id": "0c621547-8988-4acc-b15e-939a2a4174e2",
    "firstName": "Professor",
    "lastName": "Smith",
    "email": "teacher@schoolos.com",
    "role": "TEACHER",
    "isActive": true
  },
  {
    "id": "e72a819b-...",
    "firstName": "Alex",
    "lastName": "Doe",
    "email": "student@schoolos.com",
    "role": "STUDENT",
    "isActive": true
  }
]
```

PATCH `/api/v1/users/:userId/role` — ADMIN Only

Elevates or reassigns a user's role to `TEACHER`, `ADMIN`, or `STUDENT`.

REQUEST BODY

```
{
  "role": "TEACHER"
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "id": "c1f7a4e2-...",
  "firstName": "Alex",
  "lastName": "Rivera",
  "email": "alex.rivera@example.com",
  "role": "TEACHER",
  "isActive": true
}
```

PATCH `/api/v1/users/:userId/status` — ADMIN Only

Activates or deactivates a user account. Deactivated accounts are blocked from login.

REQUEST BODY

```
{
  "isActive": false
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "id": "c1f7a4e2-...",
  "email": "alex.rivera@example.com",
  "isActive": false
}
```

4. Assignments & Submissions APIs (Developer 3)

POST `/api/v1/assignments` — TEACHER (Course Owner) / ADMIN

Creates a gradable task for an owned course.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{teacherToken}}

{
  "title": "Assignment 1: Binary Search Tree",
  "description": "Implement insertion, search, and inorder traversal in TypeScript.",
  "courseId": "{{courseId}}",
  "dueDate": "2026-12-31T23:59:59.000Z",
  "maxScore": 100
}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Assignment created successfully",
  "data": {
    "id": "6a9f82d1-0322-4a11-b01c-6d9124a91fbc",
    "title": "Assignment 1: Binary Search Tree",
    "courseId": "00000000-0000-0000-0000-000000000001",
    "maxScore": 100,
    "dueDate": "2026-12-31T23:59:59.000Z",
    "createdAt": "2026-10-18T18:00:00.000Z"
  }
}
```

GET `/api/v1/assignments/course/:courseId` — Enrolled Student / Teacher / Admin

Lists all assignments published in a course. Students must be enrolled.

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "data": [
    {
      "id": "6a9f82d1-0322-...",
      "title": "Assignment 1: Binary Search Tree",
      "maxScore": 100,
      "_count": { "submissions": 1 }
    }
  ]
}
```

POST `/api/v1/assignments/:id/submissions` — STUDENT (Enrolled in Course)

Submits work for an assignment. Detects and flags late turn-ins automatically.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{studentToken}}

{
  "content": "Completed full BST tests with 100% code coverage.",
  "fileUrl": "https://github.com/schoolos/student-bst-repo"
}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Assignment submitted successfully",
  "data": {
    "id": "51a3bc89-9a72-4d1e-8bc2-a1f94892c90e",
    "assignmentId": "6a9f82d1-0322-...",
    "studentId": "e72a819b-...",
    "submittedAt": "2026-10-18T18:05:00.000Z",
    "isLate": false
  }
}
```

GET /api/v1/assignments/:id/submissions/me

STUDENT

Retrieves the logged-in student's submission, score, and teacher feedback.

HEADERS

```
Authorization: Bearer {{studentToken}}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "data": {
    "id": "51a3bc89-...",
    "score": 95,
    "feedback": "Outstanding implementation and clean test suites.",
    "submittedAt": "2026-10-18T18:05:00.000Z"
  }
}
```

GET /api/v1/assignments/:id/submissions

TEACHER / ADMIN

Lists all student submissions for an assignment. Includes student names and emails.

HEADERS

```
Authorization: Bearer {{teacherToken}}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "data": [
    {
      "id": "51a3bc89-...",
      "score": null,
      "student": {
        "id": "e72a819b-...",
        "firstName": "Alex",
        "lastName": "Doe",
        "email": "student@schoolos.com"
      }
    }
  ]
}
```

PATCH `/api/v1/assignments/submissions/:submissionId/grade` TEACHER (Course Owner) / ADMIN

Grades a submission. Automatically creates or updates the consolidated course `Grade` entry in a single atomic transaction.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{teacherToken}}

{
  "score": 95,
  "feedback": "Outstanding implementation and clean test suites."
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Submission graded successfully",
  "data": {
    "id": "51a3bc89-...",
    "score": 95,
    "feedback": "Outstanding implementation and clean test suites."
  }
}
```

5. Quizzes & Automated Assessment APIs (Developer 3)

POST `/api/v1/quizzes` — TEACHER / ADMIN

Creates a timed quiz tied to a course.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{teacherToken}}

{
  "title": "Quiz 1: Data Structures Fundamentals",
  "description": "Test your knowledge on Stacks, Queues, and Binary Search Trees.",
  "courseId": "{{courseId}}",
  "timeLimit": 30
}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Quiz created successfully",
  "data": {
    "id": "7b2e91a0-...",
    "title": "Quiz 1: Data Structures Fundamentals",
    "timeLimit": 30,
    "courseId": "00000000-0000-0000-0000-000000000001"
  }
}
```

POST `/api/v1/quizzes/:id/questions` — TEACHER / ADMIN

Adds a question to the quiz bank with options. Supports `SINGLE_CHOICE`, `MULTIPLE_CHOICE`, `TRUE_FALSE`, and `SHORT_ANSWER`.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{teacherToken}}

{
  "questionText": "What is the average time complexity of searching in a Balanced BST?",
  "questionType": "SINGLE_CHOICE",
  "points": 2,
  "options": [
    { "optionText": "O(1)", "isCorrect": false },
    { "optionText": "O(log n)", "isCorrect": true },
    { "optionText": "O(n)", "isCorrect": false }
  ]
}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Question added to quiz successfully",
  "data": {
    "id": "q1-uuid",
    "questionText": "What is the average time complexity...?",
    "points": 2,
    "options": [ ... ]
  }
}
```

POST `/api/v1/quizzes/:id/attempts/start` — STUDENT

Starts a timed attempt session. Returns questions with `isCorrect` hidden from options to prevent cheating.

HEADERS

```
Authorization: Bearer {{studentToken}}
```

EXPECTED RESPONSE (201 CREATED)

```
{
  "success": true,
  "message": "Quiz attempt started",
  "data": {
    "attemptId": "attempt-uuid-1234",
    "startedAt": "2026-10-19T15:05:00.000Z",
    "timeLimit": 30,
    "quiz": { ... }
  }
}
```

POST `/api/v1/quizzes/attempts/:attemptId/submit` — STUDENT

Submits answers, triggers the automated grading engine, and automatically syncs the student's score into the consolidated `Grade` table in an atomic transaction.

HEADERS & REQUEST BODY

```
Authorization: Bearer {{studentToken}}

{
  "answers": [
    {
      "questionId": "q1-uuid",
      "selectedOptionId": "opt2-uuid"
    }
  ]
}
```

EXPECTED RESPONSE (200 OK)

```
{
  "success": true,
  "message": "Quiz submitted and graded successfully",
  "data": {
    "attemptId": "attempt-uuid-1234",
    "score": 2,
    "maxScore": 2,
    "percentage": "100%",
    "completedAt": "2026-10-19T15:15:00.000Z"
  }
}
```

6. Step-by-Step Verification Flow

🚀 End-to-End Test Run (10 Minutes)

1. **Health Check:** Send `GET /health` ➔ Verify `{"status":"ok"}`.
2. **Teacher Login:** `POST /api/v1/auth/login` with `teacher@schoolos.com` / `Teacher123!` ➔ Save `accessToken` as `{{teacherToken}}`.
3. **Create Assignment:** `POST /api/v1/assignments` with `{{teacherToken}}` and `courseId: "00000000-0000-0000-0000-000000000001"` ➔ Save returned `id` as `{{assignmentId}}`.
4. **Student Login:** `POST /api/v1/auth/login` with `student@schoolos.com` / `Student123!` ➔ Save `accessToken` as `{{studentToken}}`.
5. **Submit Work:** `POST /api/v1/assignments/{{assignmentId}}/submissions` with `{{studentToken}}` ➔ Save returned `id` as `{{submissionId}}`.
6. **Teacher Review:** `GET /api/v1/assignments/{{assignmentId}}/submissions` with `{{teacherToken}}` ➔ View submitted work.
7. **Teacher Grade:** `PATCH /api/v1/assignments/submissions/{{submissionId}}/grade` with `{{teacherToken}}` and `{"score": 95}` ➔ Verify grade updated.
8. **Student Check:** `GET /api/v1/assignments/{{assignmentId}}/submissions/me` with `{{studentToken}}` ➔ Verify score (95) and feedback visible!

6. Standard HTTP Status Codes & Error Formats

Status Code	Meaning	Scenario
`200 OK`	Success	Query succeeded or resource updated
`201 Created`	Resource Created	User registered, assignment created, or work submitted
`400 Bad Request`	Validation Error	Invalid email, password too short, or score > maxScore
`401 Unauthorized`	Missing/Invalid Token	Missing `Bearer` token or token expired
`403 Forbidden`	Insufficient Role	Student trying to create assignment or grade submission
`404 Not Found`	Missing Resource	Assignment ID or Course ID does not exist in database
`409 Conflict`	Duplicate Operation	Student attempting duplicate submission for same assignment